Programa de Articulação da Formação Profissional Média e Superior (AMS)

Disciplina:
Estruturas de Dados

Árvores

- Prof. Alexandre Ponce de Oliveira

alexandreponceo@gmail.com

Aula 39 e 40

Lins/SP

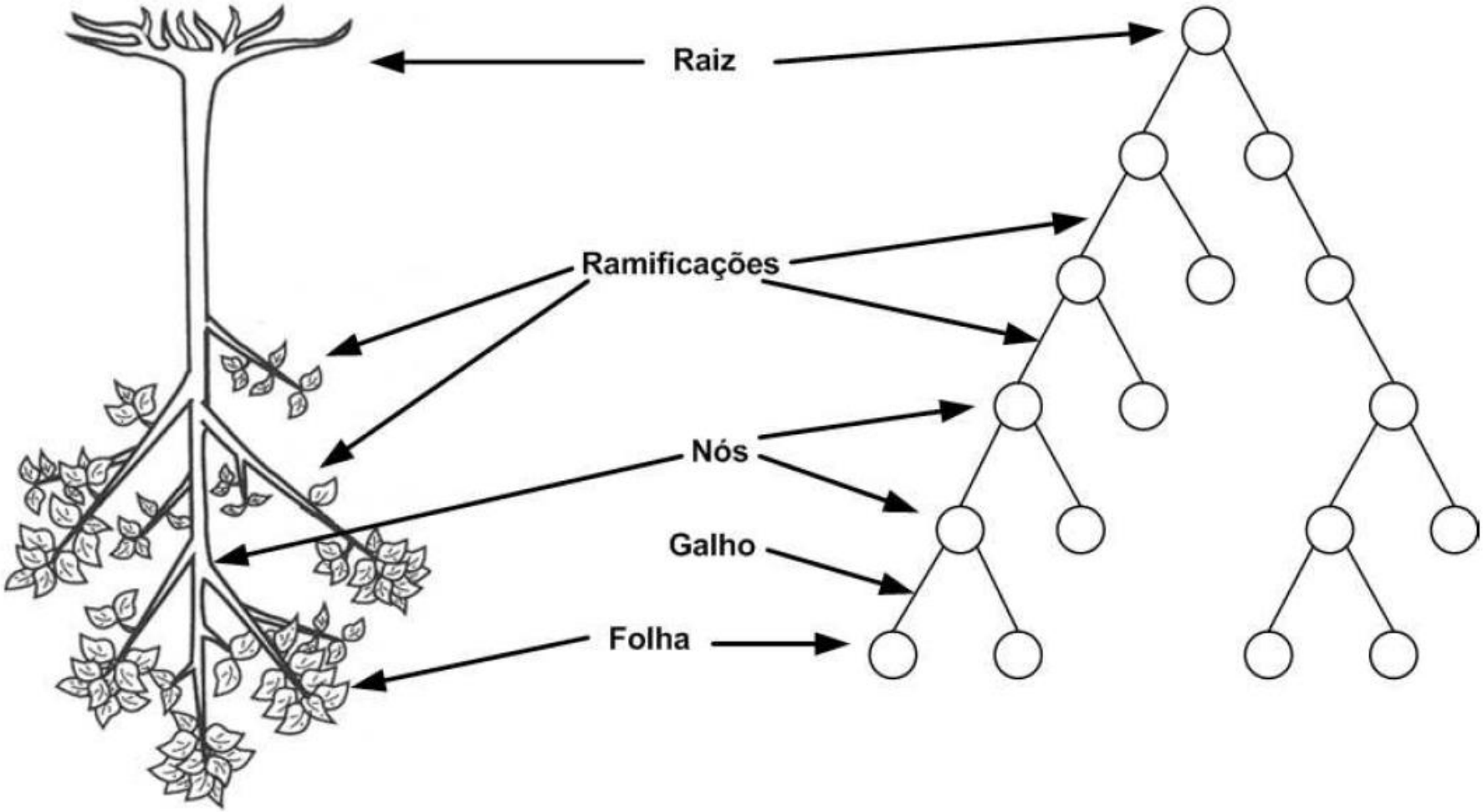
Árvores

Esquemas em árvores são utilizados para representar estruturas hierárquicas: por exemplo, árvores genealógicas.

Em computação, podem ser utilizadas para representar estrutura de diretórios, definições formais de linguagem ou representar hierarquia entre elementos.

Em ciência da computação, é uma estrutura de dados que herda as características das topologias em árvore onde os dados estão dispostos de forma hierárquica.

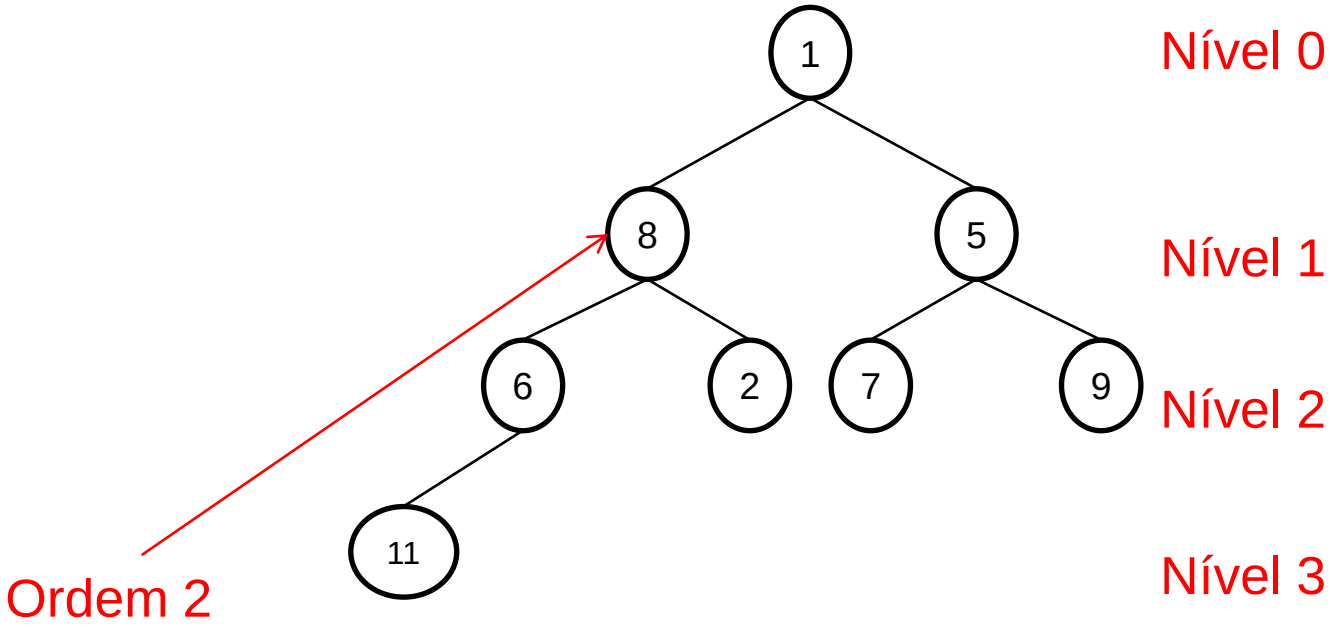
Árvores



Árvores

Grau de um nó: número de subárvores relacionadas com o nó:

- Nó 8, grau 2
- Nó 6, grau 1



Ordem: número máximo de galhos em um elemento

Altura 4

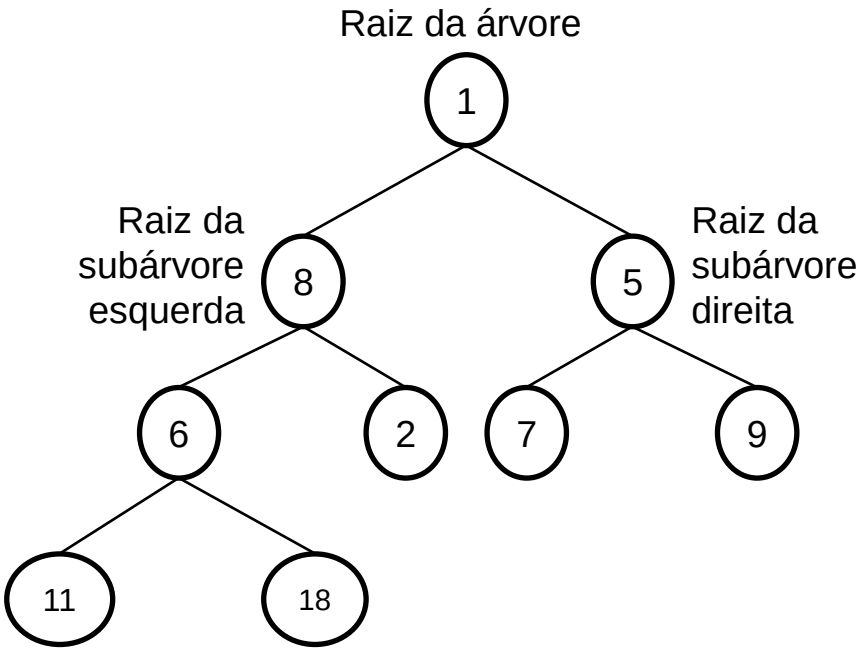
Árvores Binárias

Conjunto finito de elementos que está vazio ou é particionado em três subconjuntos:

- raiz da árvore - elemento inicial (único);

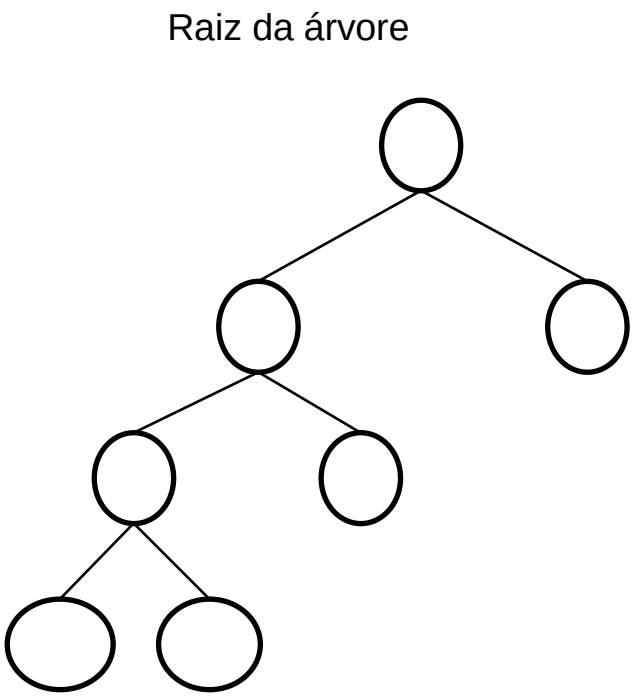
- subárvore da esquerda;

- subárvore da direita.



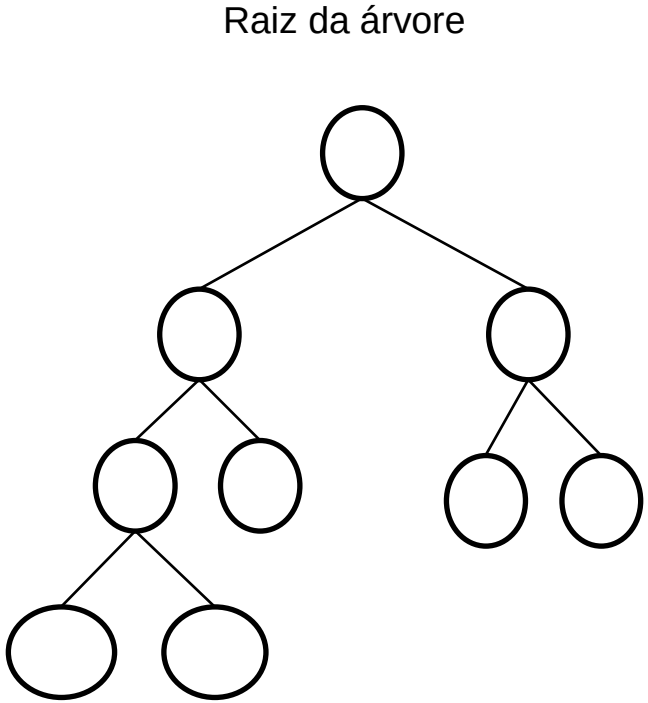
Árvores Binárias

a) Árvore estritamente binária: nó tem 0 ou 2 filhos;



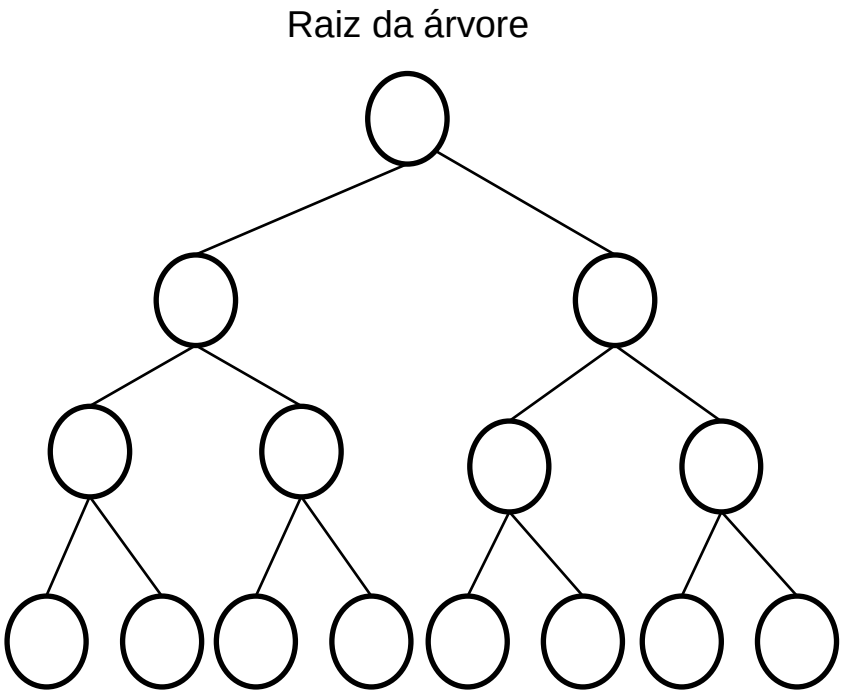
Árvores Binárias

b) Árvore binária completa: é aquela em se n é um nó com algumas subárvores vazias, então n se localiza no penúltimo ou no último nível.



Árvores Binárias

c) Árvore binária cheia: se o nó tem alguma subárvore vazia então ele está no último nível.

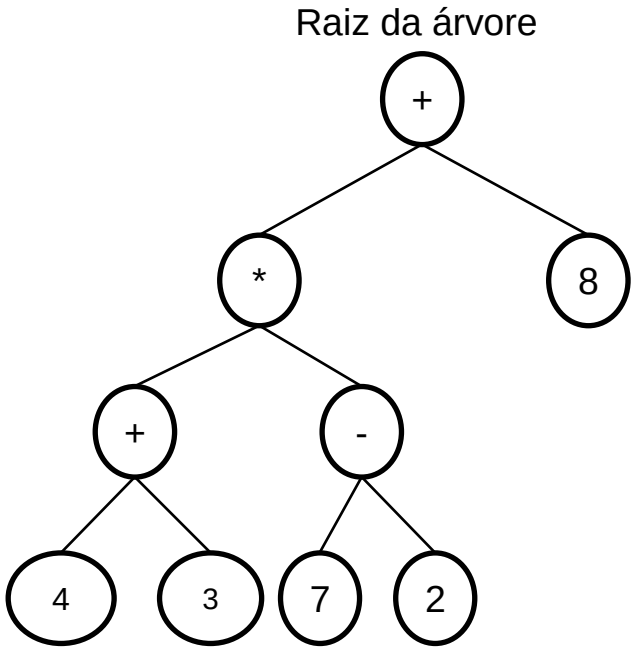


Árvores Binárias

Exemplo: (4 + 3) * (7 - 2) + 8

Expressões aritméticas com o uso de árvores binárias:

- folhas representam operandos
- nós operadores



Estrutura das Árvores Binárias

```
struct No {
        int valor;
        struct No *esq;
        struct No *dir;
};
typedef struct No arvore;
```

Árvores Binárias

Funções em C:

- Inicializa
- Insere
- Mostra
- Exclui
- Busca

Árvores Binárias

```
arvore* inicializa(void)
{
        return NULL;
}

arvore* insere(arvore* esq, arvore* dir)
{
        arvore *nova = (arvore *) malloc(sizeof(arvore));
        if(!nova)
        {       printf("Impossivel alocar memoria\n");
                exit(1);    }
        printf("Digite um valor para inserir na arvore: ");
        scanf("%d", &nova->valor);
        nova -> esq = esq;
        nova -> dir = dir;
        return nova;
}
```

Árvores Binárias

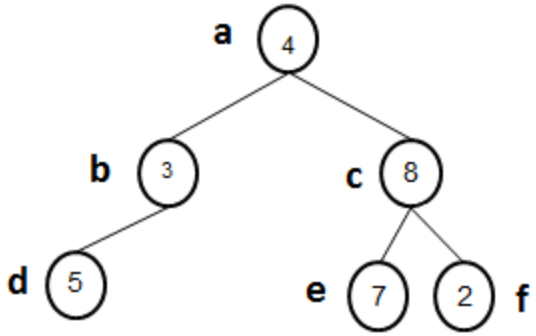
```
void mostra(arvore* Arvore)
{       if (Arvore != NULL)
        {       printf(" %d  ", Arvore->valor);
                mostra(Arvore->esq);
                mostra(Arvore->dir);
        }
}

arvore* exclui(arvore* Arvore)
{       if (Arvore != NULL)
        {       exclui(Arvore->esq);
                exclui(Arvore->dir);
                free(Arvore);
        }
        return NULL;
}
```

Árvores Binárias

```
void busca (arvore* Arvore, int x)
{
        if (Arvore == NULL)
                return 0;
        else
                if (x == Arvore->valor)
                        printf("Encontrado!");
                else
                {
                        busca(Arvore->esq,x);
                        busca(Arvore->dir,x);
                }
}
```

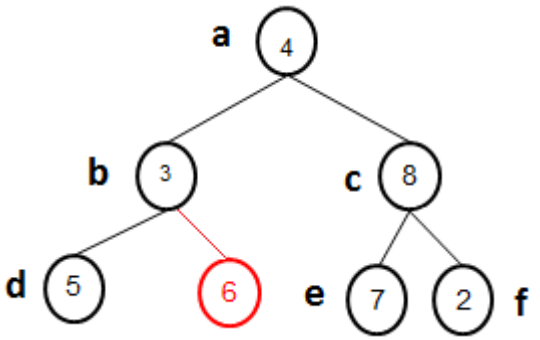
Árvores Binárias




```
int main(void)
{
        int i;
        arvore* d = insere(inicializa(), inicializa());
        arvore* b = insere(d, inicializa());
        arvore* e = insere(inicializa(), inicializa());
        arvore* f = insere(inicializa(), inicializa());
        arvore* c = insere(e, f);
        arvore* a = insere(b, c);
        mostra(a);
        a = exclui(a);
        return 0;
}
```

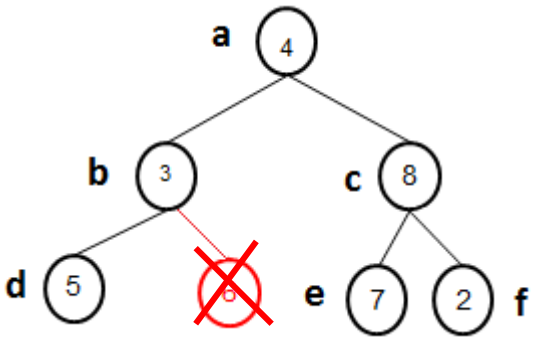
Árvores Binárias – Inserção de elementos

```
a->esq->dir = insere(inicializa(),inicializa());
mostra(a);
```



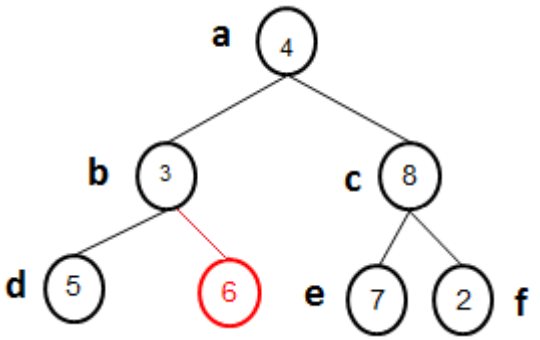
Árvores Binárias – Exclusão de elementos

a->esq->dir = exclui(a->esq->dir);



Árvores Binárias – Busca de elementos

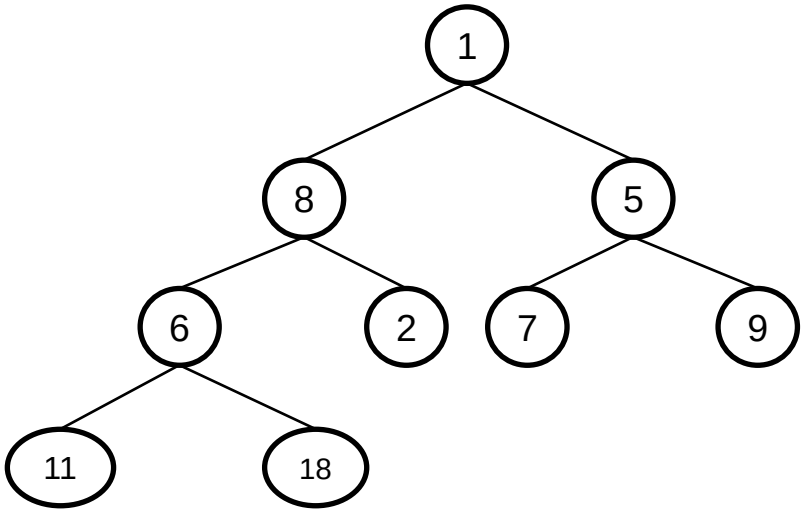
```
printf("Digite um valor para procurar na arvore: ");
scanf("%d", &i);
busca(a,i);
```



Exercícios

1) Escreva uma função que percorre uma árvore binária para determinar sua altura.

2) Reescreva a função main() para gerar uma árvore binária ilustrada pela figura abaixo:



Exercícios

3) Reescreva a função main() para gerar uma árvore binária ilustrada pela figura abaixo:

